DEADLOCKS

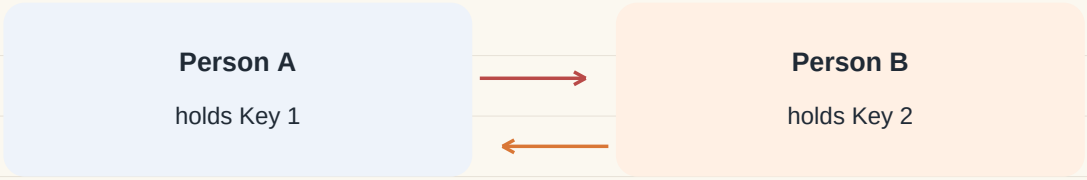
DAY 02 / 50 • BACKEND ENGINEERING

When transactions wait on each other, progress can stop.

THE IDEA

A deadlock occurs when transactions each hold a resource and wait for another resource held by someone else. The cycle prevents either transaction from proceeding.

A simple analogy



Each waits for the key held by the other.

Remember

Race condition: an unsafe outcome from concurrent work. Deadlock: a cycle of waiting.

How a deadlock forms

Two transactions need the same two accounts, but they lock them in opposite order.

01 **Transaction A**

Locks Account 101

↓

02 **Transaction B**

Locks Account 202

↓

03 **Transaction A**

Requests Account 202 — waits

↓

04 **Transaction B**

Requests Account 101 — waits

DEADLOCK DETECTED

The database detects the wait cycle and aborts one transaction. The other can continue after the conflict is cleared.

Prevent it before it happens

1. Lock resources in a consistent order

For example, always lock Account 101 before Account 202.

2. Keep transactions short

Do minimum work while holding locks. Avoid slow API calls inside a transaction.

3. Use appropriate indexes

Indexes may reduce scanning and lock duration, depending on query and database.

4. Recover safely

Roll back and retry the entire transaction a bounded number of times.

Important

Consistent lock order reduces risk but cannot guarantee that every deadlock disappears. Handle database errors.

SQL: lock in a stable order

Illustrative PostgreSQL-style pattern for a transfer touching two account rows:

```
BEGIN;

SELECT id, balance
FROM accounts
WHERE id IN (101, 202)
ORDER BY id
FOR UPDATE;

-- Validate balances and update both rows
-- inside this same transaction.

COMMIT;
```

Why ORDER BY?

It encourages a consistent row-locking order. Keep balance checks and updates in the same transaction.

Production note: check that both rows exist, enforce sufficient funds, roll back on errors, and retry only transient deadlock failures. This is a pattern, not a complete transfer implementation.

The takeaway

Deadlock = circular waiting

Transactions hold locks that the other transactions need, so the database must break the cycle.

Quick review

- [ ] Can I explain race condition vs deadlock?
- [ ] Do related transactions lock shared rows in a consistent order?
- [ ] Are transactions short, with no slow external calls while holding locks?
- [ ] Can the app roll back and safely retry the whole transaction when appropriate?

Learning in public

Day 2/50 — learning the engineering behind reliable backend systems, one topic at a time.

Learning notes only. SQL is illustrative, not a complete transfer implementation.